Huellitas Saludables PRO — reglas de negocio, arquitectura y decisiones técnicas

Versión: 1.0
Fecha de revisión: 13 de noviembre de 2026
Alcance: frontend, API/backend, persistencia PostgreSQL, seguridad, calidad y plan de evolución.
Estado: documento maestro basado en el código y las migraciones presentes en esta copia del repositorio.

> Este documento consolida las 45 reglas del archivo `Reglas_de_Negocio_RN.docx`, añade cinco reglas complementarias identificadas en el código/bitácoras (RN46–RN50) y contrasta el `Plan_18_Tareas_Huellitas_Saludables_Detallado.pdf` con la implementación. Distingue lo que ya existe de lo parcial, sustituido o futuro; un punto que aparece en un plan no se considera implementado hasta encontrar evidencia en el código.

1. Resumen ejecutivo

Huellitas Saludables es una aplicación web de gestión veterinaria para usuarios propietarios de mascotas, recepcionistas, especialistas y administradores. La solución está organizada como un **monolito modular de tres capas**: una interfaz web React, una API Node.js/Express y una base de datos relacional PostgreSQL. Frontend y backend se ejecutan como servicios distintos en Docker Compose, pero el backend es una única aplicación con módulos y responsabilidades separadas; no es una arquitectura de microservicios.

El backend usa JavaScript con módulos ESM, Prisma ORM 7 y PostgreSQL 16. El frontend usa React 18, TypeScript, Vite, React Router y Axios. La base cuenta hoy con ocho modelos Prisma y SQL de migración que agrega restricciones, un índice parcial y triggers que deben conservarse al evolucionar el esquema.

El documento adjunto contiene 45 reglas; se agregan cinco reglas complementarias (RN46–RN50) para cubrir la validación temporal, la cancelación automática, su auditoría y la conservación del historial implementadas después. La intención de varias reglas coincide con el código, pero hay diferencias importantes: la autenticación vigente usa JWT Bearer, no los headers `x-user-id` / `x-user-role`; el registro de auditoría persistente cubre actualmente cancelaciones, no todas las acciones administrativas; y el JSON de error contiene `code` y `details`, no siempre una propiedad `data`. Las tareas del plan también tienen distintos estados de avance, reflejados más adelante.

2. Fuentes, alcance y criterio de evidencia

Se revisaron, entre otros, `back/prisma/schema.prisma`, las migraciones Prisma, `back/app/`, `back/package.json`, `front/package.json`, `front/src/`, `docker-compose.yml`, `Readme.md`, las bitácoras del proyecto y los dos documentos adjuntos.

La prioridad de interpretación es:

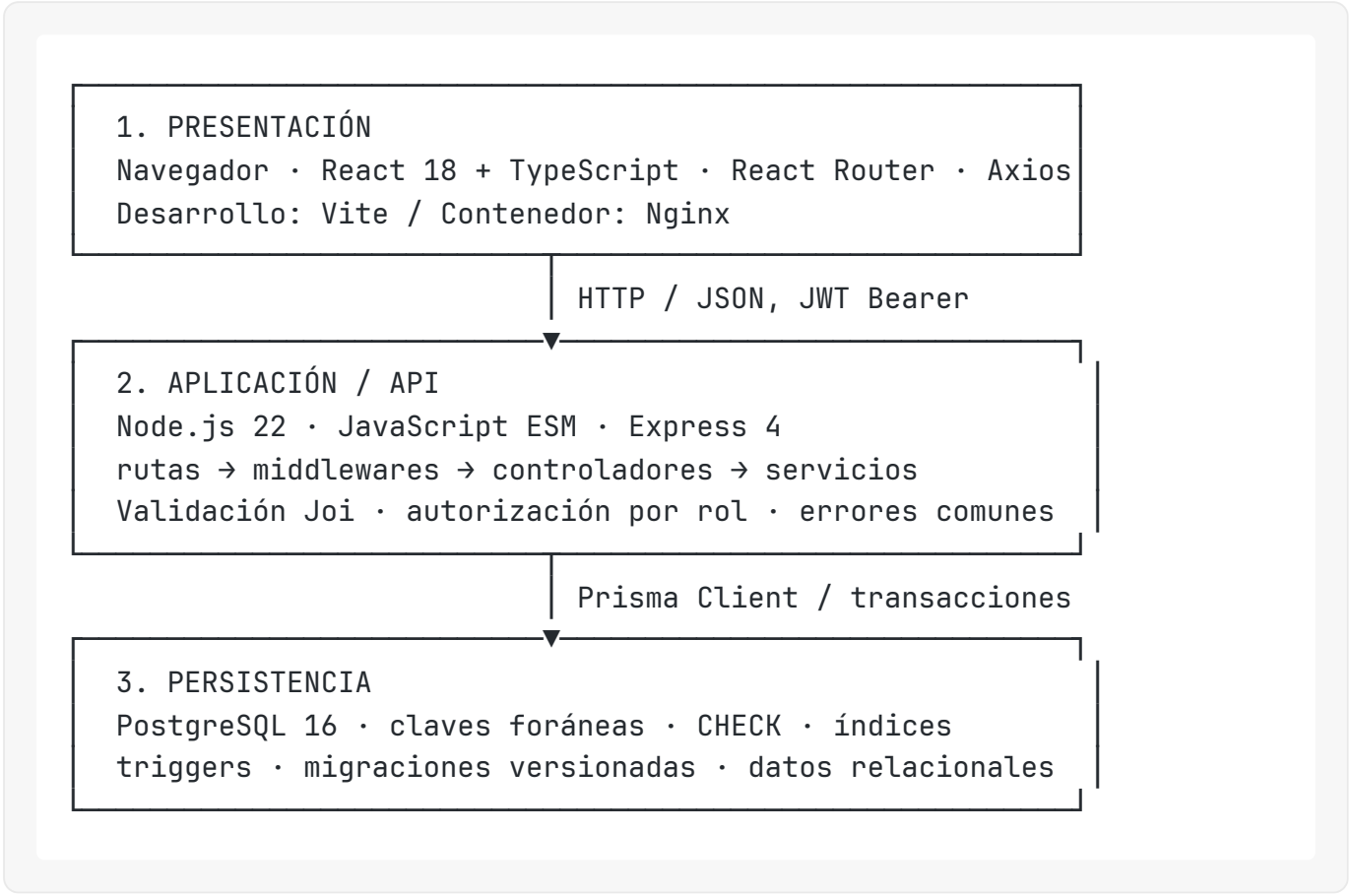
1. **Código, esquema Prisma y migraciones vigentes:** describen el comportamiento implementado.
2. **Pruebas existentes y bitácoras fechadas:** apoyan la evidencia de casos probados; no sustituyen una comprobación de ejecución actual.
3. **Reglas de negocio y plan adjuntos:** describen requisitos y trabajo propuesto; pueden necesitar actualización si difieren del código.
4. **Justificaciones de diseño:** cuando el repositorio no conserva una decisión histórica, se presentan como razones técnicas para mantener la solución observada, no como afirmaciones de lo que el equipo discutió en su momento.

Estados usados:

- **Implementada:** existe evidencia directa en rutas, servicios, middleware, esquema, migración o interfaz.
- **Parcial:** existe una parte del requisito, pero falta alcance funcional, persistencia, interfaz o endurecimiento.
- **Actualizada/sustituida:** el objetivo de la regla se conserva, pero el mecanismo original ya no corresponde al sistema actual.
- **Pendiente:** no se encontró implementación suficiente.
- **Requiere revisión:** hay una brecha de seguridad, privacidad o consistencia documental que debe decidir el equipo.

3. Arquitectura actual

3.1 Vista de tres capas



La interfaz llama a la API por HTTP. En Docker, Nginx sirve los archivos compilados de la SPA y reenvía `/api`, `/health` y `/api-docs` al backend. En desarrollo, Vite se ejecuta por separado y la API se configura con `VITE_API_URL` o `http://localhost:3000/api`.

3.2 Monolito modular, no microservicios

La API se despliega como un servicio backend único, pero separa transporte, reglas de negocio y persistencia en módulos. Esta forma reduce la complejidad operativa de una clínica con un dominio relacionado y permite que una operación transaccional —por ejemplo, reservar una cita y ocupar una franja— permanezca dentro de una sola aplicación y una sola base. Docker Compose separa frontend, backend y base de datos como servicios de ejecución; eso no convierte la API en microservicios.

Estructura principal del backend:

Responsabilidad	Ubicación	Función
Rutas	`back/app/routes/`	Endpoints HTTP y composición de validación/permisos.
Middleware	`back/app/middlewares/`	JWT, roles, Joi, auditoría y tratamiento común de errores.
Controladores	`back/app/controllers/`	Traducción entre HTTP y operaciones de aplicación.
Servicios	`back/app/services/`	Reglas de negocio, autorización por recurso y transacciones coordinadas.
Modelos/adaptadores	`back/app/models/`	Consultas Prisma y adaptación de resultados al contrato de la API.
Configuración/utilidades	`back/app/config/`, `back/app/utils/`	Cliente Prisma, Swagger, JWT, fechas, hashing, logger y utilidades seguras.
Esquema y migraciones	`back/prisma/`	Modelos Prisma, historial de cambios SQL y semilla.

En el frontend, `pages/` organiza pantallas; `components/` contiene piezas reutilizables; `routes/` define navegación y guards; `context/` mantiene el contexto de autenticación; `services/api.ts` concentra Axios.

4. Stack y decisiones técnicas

DT-01 — JavaScript ESM y Node.js para el backend

Decisión observada: Node.js 22 y JavaScript con ES Modules. `back/package.json` configura `type: module`; los módulos emplean `import` / `export` y la imagen Docker usa Node 22.

Razón técnica para mantenerlo: el backend ya está implementado en este ecosistema y su carga principal es recibir solicitudes, validar datos y esperar operaciones de PostgreSQL y red

(trabajo mayormente de I/O). El modelo asíncrono de Node.js es adecuado para esa clase de servicio y evita duplicar la API en otro runtime. El mismo ecosistema npm reúne Express, Prisma, Joi, JWT, bcrypt, Vitest y Supertest.

Rendimiento, expresado con precisión: esto no demuestra que JavaScript o Node sean siempre más rápidos que Python/FastAPI, ni sustituye una medición. El rendimiento final depende de consultas, índices, conexiones, carga, serialización y despliegue. Node requiere evitar trabajo síncrono largo o cálculo intensivo en el event loop; si el proyecto alcanza ese tipo de carga, debe perfilarse y considerar workers o servicios especializados.

DT-02 — React y TypeScript para la interfaz

Decisión observada: React 18, TypeScript, Vite, React Router y Axios. La aplicación es una SPA con rutas, layouts, componentes, contexto de autenticación y páginas por función.

Razón técnica para mantenerlo: las pantallas comparten navegación, sesión, formularios y patrones de interfaz. React permite componer componentes y actualizar la interfaz a partir del estado; React Router resuelve navegación y guards; TypeScript ayuda a expresar contratos del frontend; Vite ofrece un flujo de desarrollo y compilación integrado con el proyecto. El stack existente permite continuar y reutilizar pantallas en vez de reescribirlas.

Rendimiento: React facilita actualizaciones acotadas mediante su modelo de renderizado, pero no garantiza por sí solo que la aplicación sea más rápida. No hay pruebas de carga del frontend ni benchmark comparativo en el repositorio. Vite mejora principalmente el flujo de desarrollo/compilación; no debe confundirse la rapidez del servidor de desarrollo con el rendimiento final percibido.

Estado de calidad de frontend: no se encontraron pruebas automatizadas de frontend ni suite E2E en la revisión estática. El comando de build declarado es `tsc -b && vite build`; la compilación se reportó exitosa en la verificación documentada de la modificación anterior. La auditoría no ejecutó de nuevo el build en este documento.

Aclaración importante: React no se compara con FastAPI

React es una biblioteca de interfaz; FastAPI es un framework de Python para crear APIs backend. No son alternativas del mismo nivel. FastAPI **no traduce ni convierte JavaScript a otro lenguaje**: es un framework distinto que permite implementar el servidor con Python. React podría seguir siendo el frontend aunque el backend se escribiera con FastAPI.

Migrar la API actual a FastAPI requeriría reimplementar en Python rutas, autenticación, validaciones, acceso a datos, pruebas y operación, y mantener una frontera HTTP entre el frontend y el backend. Solo tendría sentido si hubiera una necesidad concreta —por ejemplo, una integración que favorezca Python— y el beneficio medido compensara la reescritura. No hay evidencia en los archivos revisados de que FastAPI se haya descartado tras una comparación formal.

DT-03 — Express como framework HTTP

Express organiza las rutas y middlewares de la API. Se usa para montar JSON, CORS, healthcheck, Swagger, identificación JWT, auditoría y manejador común de errores. Su flexibilidad encaja con el backend modular; como no impone por sí solo una arquitectura, el proyecto debe conservar convenciones claras para controladores, servicios, validación y permisos.

DT-04 — PostgreSQL 16 para persistencia relacional

El dominio relaciona usuarios, propietarios, mascotas, razas, especialistas, horarios, citas, cancelaciones e historias clínicas. PostgreSQL se usa por sus relaciones, claves foráneas, restricciones, transacciones, índices y soporte de SQL necesario para las reglas del agendamiento. En Docker Compose se configura `postgres:16-alpine`.

Las validaciones de interfaz o API no sustituyen la integridad de la base. La BD mantiene una última barrera ante escrituras inconsistentes, incluso si una operación no pasa por una pantalla determinada.

DT-05 — Prisma ORM 7 ESM como capa de acceso a datos

Prisma proporciona el esquema declarativo, el cliente JavaScript/ESM y las operaciones relacionadas con PostgreSQL. El proyecto usa `@prisma/adapter-pg`, un cliente singleton en `back/app/config/prisma.js`, `back/prisma/schema.prisma` y Prisma Migrate.

Por qué resulta conveniente aquí: concentra modelos y relaciones, ofrece consultas desde el mismo lenguaje del backend, facilita las operaciones CRUD habituales y organiza migraciones. Las transacciones permiten agrupar escrituras relacionadas. Esto reduce código repetitivo de acceso a datos y facilita mantener sincronizados el esquema y la aplicación.

Límites: un ORM no garantiza que toda consulta sea óptima ni reemplaza SQL cuando hace falta. El proyecto conserva SQL parametrizado y SQL de migración para `FOR UPDATE`, triggers, restricciones `CHECK` e índice único parcial que no se expresan por completo en el schema Prisma. Deben mantenerse y probarse conjuntamente.

DT-06 — Modelo de seguridad de API

- **Autenticación:** JWT real firmado HS256, enviado como `Authorization: Bearer <token>`. El middleware busca al usuario vigente en PostgreSQL y obtiene de allí su rol; no confía en un rol escrito libremente por el cliente.
- **Autorización:** middlewares `requireRole` y comprobaciones por recurso en servicios. La protección de rutas de React mejora la experiencia, pero la frontera de seguridad es el backend.

- **Contraseñas:** bcrypt para nuevos hashes; valor predeterminado de 12 rondas. Se acepta verificar un hash scrypt heredado para migrarlo a bcrypt tras autenticación válida; contraseñas en texto plano no se aceptan en login.
- **Tokens:** el cierre de sesión limpia el estado del cliente, pero no se encontró revocación inmediata en servidor. Un JWT emitido puede ser válido hasta expirar salvo otros controles; la invalidación de tokens sigue pendiente.
- **Validación:** Joi en backend y controles de formulario en frontend. El servidor vuelve a validar siempre; las reglas de entrada actuales incluyen, entre otras, teléfono numérico y correo/contraseña sin espacios.
- **Errores y logs:** un único middleware transforma errores HTTP; fallos internos muestran texto genérico al cliente. Pino define niveles por entorno y redacta tokens, credenciales y datos personales.

DT-07 — Integridad y concurrencia de citas en la base de datos

Las reservas usan transacciones y SQL parametrizado con bloqueo de fila `FOR UPDATE`. Un índice único parcial protege frente a dos citas activas en una misma franja; una restricción `UNIQUE` impide duplicar fecha/hora para un especialista. Los triggers sincronizan estado de disponibilidad y el estado de la cita al insertar historia clínica. Las reglas se implementan en varios niveles porque la operación debe seguir siendo correcta ante solicitudes concurrentes.

DT-08 — Pruebas y documentación de API

El backend usa Vitest y Supertest para pruebas unitarias e integración. La configuración de pruebas está diseñada para usar una base con sufijo `_test`; la inicialización la elimina y la crea de nuevo, por lo que se debe comprobar `DB_NAME_TEST` antes de ejecutar tests. Swagger/OpenAPI se sirve en `/api-docs`, pero su especificación actual tiene descripciones heredadas de `x-user-id`, `x-user-role` y `mock-token`; debe alinearse con JWT Bearer antes de usarla como contrato confiable.

DT-09 — Docker Compose y configuración por entorno

Docker Compose coordina PostgreSQL, backend y frontend. Nginx sirve los activos de la SPA y funciona como proxy hacia la API. La solución admite desarrollo y demostración local; el README del proyecto advierte que el despliegue de producción aún no está definido.

Hay un riesgo de configuración que debe corregirse antes de producción: el código exige `JWT_SECRET`, pero `docker-compose.yml` puede inyectar un secreto placeholder; `DB_PASSWORD` también tiene defaults. Además, Compose establece `CORS_ORIGIN=*`, mientras Express trata el valor como una lista literal de orígenes separados por coma; no es

una configuración de lista explícita y puede rechazar orígenes del navegador. Usar secretos fuertes y una lista exacta por entorno.

5. Base de datos: estado actual y evolución prevista

5.1 Modelos actuales

El schema vigente contiene **ocho modelos**. Siete son las entidades funcionales base y el octavo incorpora auditoría persistente de cancelaciones.

Modelo/tabla	Propósito y relación principal
`Usuario` / `usuario`	Cuenta, rol, contacto y especialización opcional. Correo único.
`Raza` / `raza`	Catálogo de razas; una raza puede asociarse a varias mascotas.
`Mascota` / `mascota`	Datos de la mascota y referencia a una raza; tiene propietarios y citas.
`UsuarioMascota` / `usuario_mascota`	Tabla puente de muchos-a-muchos entre usuario y mascota; permite modelar propietario principal y acudientes.
`Disponibilidad` / `disponibilidad`	Franja de especialista, con fecha, hora y estado; no duplica especialista/fecha/hora.
`Cita` / `cita`	Relaciona mascota, disponibilidad y recepcionista; guarda motivo y estado.
`CitaCancelacion` / `cita_cancelacion`	Una auditoría por cita cancelada: actor, rol, origen, motivo y fecha; no borra la cita.
`HistoriaClinica` / `historia_clinica`	Registro clínico asociado a una cita; máximo una historia por cita.

Relaciones principales: Usuario–Mascota es muchos-a-muchos por `UsuarioMascota`; Raza–Mascota uno-a-muchos; Usuario especialista–Disponibilidad uno-a-muchos; Mascota–Cita uno-a-muchos; Disponibilidad–Cita uno-a-muchos en historial, con máximo una cita activa por franja; Cita–HistoriaClinica uno-a-cero/uno; Cita–CitaCancelacion uno-a-cero/uno.

5.2 Reglas almacenadas directamente en PostgreSQL

Las migraciones contienen claves foráneas y `CHECK` para rol, tipo de propietario, especie, género, estado de disponibilidad, estado de cita, origen de cancelación y peso positivo. También contienen:

- Correo único y unicidad de especialista/fecha/hora en disponibilidad.
- Unicidad de historia por cita y una auditoría de cancelación por cita.
- Índice único **parcial** para que una disponibilidad tenga como máximo una cita no cancelada.
- Triggers que mantienen la disponibilidad y el estado de cita sincronizados.
- Restricciones de borrado `RESTRICT` para preservar referencias a citas e historias.

`back/prisma/schema.prisma` es la fuente legible de entidades y relaciones, pero no representa por completo todos los objetos SQL personalizados. Antes de tocar una relación o estado, crear una migración y comprobar que los `CHECK`, índices, triggers y claves foráneas permanezcan.

5.3 Base de datos prevista: qué está definido y qué no

El plan adjunto plantea recuperación de contraseña por correo, edición de perfiles de usuario/mascota, historias clínicas desde la vista especialista y seguimiento de mascotas por cuidadores. **No define un esquema futuro cerrado** para esas funciones; por lo tanto, este documento no afirma que ya existan tablas nuevas para ellas.

Cuando se apruebe el alcance, las extensiones posibles que deben diseñarse —no son modelos implementados— incluyen:

- **Recuperación de contraseña:** mecanismo con token no reutilizable, almacenamiento seguro (idealmente del hash del token), expiración y fecha de consumo; integrar migración/versionado con Prisma.
- **Perfil de usuario/mascota:** primero confirmar qué campos se pueden editar y por quién. Si bastan columnas actuales, quizá no se requiera tabla nueva; cualquier columna o relación nueva necesita migración.
- **Seguimiento del cuidador:** definir antes el rol del cuidador, quién puede leer/escribir, qué eventos se registran y retención. Luego diseñar una entidad de seguimiento vinculada a mascota y actor, con marcas de tiempo y límites de acceso. El nombre y campos definitivos quedan pendientes de requisitos.
- **Historias desde la vista especialista:** el backend ya expone operaciones clínicas; completar la interfaz no exige necesariamente un modelo nuevo si el flujo funcional no cambia.

Toda evolución de esquema se debe versionar en `back/prisma/migrations/`, probar contra una base aislada y preservar el historial clínico y las auditorías. No se recomienda borrado físico automático de citas/franjas antiguas: hoy se conserva el historial y el proceso automático cancela citas vencidas.

6. Reglas de negocio RN01–RN50

Las reglas RN01–RN45 y sus prioridades se reproducen conceptualmente desde `Reglas_de_Negocio_RN.docx`; el estado y observaciones de implementación se añaden a partir del repositorio revisado. “RN actualizada” indica que debe corregirse el documento original para reflejar el contrato vigente. RN46–RN50 son reglas adicionales detectadas en código y bitácoras; se proponen como continuidad del catálogo.

6.1 Usuarios y roles

ID	Prioridad	Regla de negocio	Estado observado / evidencia
RN01	Alta	Solo usuarios se autorregistran; las cuentas administrativas las crea un administrador.	**Implementada.** `POST /auth/registro` es público; `registro-admin` requiere `admin`.
RN02	Media	Cada usuario tiene un único rol activo.	**Implementada.** El modelo almacena un campo `rol`; migración limita valores.
RN03	Alta	El correo electrónico es único.	**Implementada.** Restricción e índice único en PostgreSQL.
RN04	Alta	La contraseña tiene al menos 4 caracteres.	**Implementada.** Esquema Joi de autenticación. Las restricciones de no espacios se suman al requisito.
RN05	Alta	El anexo pide `x-user-id` y `x-user-role` en rutas protegidas.	**Sustituida; actualizar regla.** El contrato vigente es `Authorization: Bearer <JWT>`. No autenticar con headers de identidad arbitrarios.
RN06	Alta	El rol del header debe coincidir con el almacenado.	**Actualizada.** El middleware verifica el JWT, vuelve a leer usuario/rol en BD y autoriza con el rol vigente; no depende de `x-user-role`.

6.2 Mascotas

ID	Prioridad	Regla de negocio	Estado observado / evidencia
RN07	Alta	Especialistas y recepcionistas no registran mascotas; lo hacen usuarios y administradores.	**Implementada.** POST de mascotas restringido a `usuario` y `admin`.
RN08	Media	La mascota se asigna automáticamente al propietario autenticado.	**Implementada, mecanismo actualizado.** El servicio deriva el propietario del usuario autenticado por JWT, no del header `x-user-id`.
RN09	Media	La raza debe existir en el catálogo.	**Implementada.** Se comprueba en servicio y la FK protege la escritura.
RN10	Alta	Solo se permiten especies perro o gato.	**Implementada.** Validación de servicio y `CHECK` SQL.
RN11	Media	Solo se permiten géneros macho o hembra.	**Implementada.** Validación de servicio y `CHECK` SQL.
RN12	Media	Una mascota puede tener propietario principal y acudientes.	**Parcial.** `UsuarioMascota` representa varios propietarios; el flujo/API completa de administración de acudientes no quedó confirmado en esta revisión.

6.3 Disponibilidad

ID	Prioridad	Regla de negocio	Estado observado / evidencia
RN13	Alta	Solo recepcionistas y administradores crean disponibilidad.	**Implementada.** POST/PUT/DELETE restringidos por rol.
RN14	Alta	La disponibilidad solo se crea para un especialista existente.	**Implementada.** El servicio verifica rol `especialista`.
RN15	Alta	Un especialista no duplica fecha y hora.	**Implementada.** Unicidad en PostgreSQL/Prisma y manejo del conflicto.
RN16	Alta	No cambiar fecha, hora o especialista si la disponibilidad está ocupada.	**Implementada.** La actualización rechaza cambios de esos campos en estado ocupado.
RN17	Alta	No eliminar disponibilidad ocupada o referenciada por cita.	**Implementada.** El servicio y la FK previenen el borrado con cita asociada.

6.4 Citas

ID	Prioridad	Regla de negocio	Estado observado / evidencia
RN18	Alta	Especialistas no crean citas; sí usuarios, recepción y administradores.	**Implementada.** Roles permitidos en POST `/api/citas`.
RN19	Alta	La mascota de una cita debe existir.	**Implementada.** Verificación de servicio y FK.
RN20	Alta	Un usuario solo agenda para una mascota que le pertenece.	**Implementada.** El servicio verifica propiedad para rol `usuario`.
RN21	Alta	La disponibilidad debe existir, estar libre y ser futura.	**Implementada.** Servicio, transacción, restricción de base e índice parcial. La hora se valida para `America/Bogota`.
RN22	Media	La cita asigna automáticamente un recepcionista.	**Implementada con precisión de roles.** Si quien crea es recepción, se usa ese usuario; para los demás roles permitidos se selecciona un recepcionista registrado.
RN23	Alta	Usar bloqueo pesimista para evitar doble reserva concurrente.	**Implementada.** Bloqueo `FOR UPDATE`, transacción y protección adicional por índice único parcial.
RN24	Alta	Un usuario no modifica ni cancela citas de otros propietarios.	**Implementada.** La propiedad se verifica para usuarios en servicios de edición/cancelación.
RN25	Media	Recepcionista/admin gestionan citas sin restricción de propiedad.	**Implementada.** Las rutas permiten ambos roles; los controles de propiedad se aplican a rol usuario.
RN26	Alta	No cancelar dos veces la misma cita.	**Implementada.** El servicio rechaza cancelación repetida; la auditoría también es única por cita.

ID	Prioridad	Regla de negocio	Estado observado / evidencia
RN27	Media	Cancelar libera la disponibilidad.	**Implementada con excepción que debe explicitarse.** La cancelación manual de una cita futura puede liberar el horario. La cancelación automática de una cita vencida conserva la franja histórica y no la reabre.

6.5 Historia clínica

ID	Prioridad	Regla de negocio	Estado observado / evidencia
RN28	Alta	Solo especialistas acceden a historias clínicas.	**Implementada.** Rutas clínicas requieren rol `especialista`.
RN29	Alta	La cita debe existir para crear historia.	**Implementada.** Servicio valida la cita y la FK protege la referencia.
RN30	Alta	Solo el especialista asignado a la cita puede crear historia.	**Implementada.** La operación transaccional verifica la asignación.
RN31	Alta	Una cita cancelada no puede tener historia nueva.	**Implementada.** Servicio rechaza la cita cancelada.
RN32	Alta	Máximo una historia por cita.	**Implementada.** Campo único y restricción de PostgreSQL.
RN33	Media	Al crear historia, la cita pasa a atendido.	**Implementada.** Trigger SQL en la migración inicial.
RN34	Alta	No editar historia clínica creada por otro especialista.	**Implementada.** El modelo/servicio verifica el especialista asociado.
RN35	Media	No eliminar historias para conservar trazabilidad.	**Implementada.** No hay endpoint de borrado de historia; FK `RESTRICT` protege la relación.

6.6 Seguridad y auditoría

ID	Prioridad	Regla de negocio	Estado observado / evidencia
RN36	Alta	Rutas API autenticadas, salvo login, registro y healthcheck.	**Implementada con contrato actualizado.** Middleware global protege `/api`; se exceptúan login/registro y `/health`.
RN37	Alta	Solo login, registro público y healthcheck son accesibles sin autenticación.	**Implementada, con revisar Swagger.** El código aplica JWT; la especificación Swagger aún muestra headers/mock heredados.
RN38	Alta	Auditar acciones de creación, edición y eliminación de recepción/admin.	**Parcial.** Las cancelaciones se auditan en `cita_cancelacion`. `auditLog.js` registra ciertos eventos estructurados solo fuera de producción; no existe auditoría general persistente de todas las acciones.
RN39	Media	Errores JSON uniformes con `{ success, message, data }`.	**Formato central implementado, forma actualizada.** Los errores incluyen `{ success:false, message, code, details }`; validaciones pueden detallar campos. Corregir la regla original para reflejar el contrato real.
RN40	Media	Respuestas API con estructura unificada.	**Implementada.** Utilidad común de respuesta, con `success`, `message`, `data` y metadatos cuando aplica; los errores siguen la forma descrita en RN39.

6.7 Endpoints y permisos

ID	Prioridad	Regla de negocio	Estado observado / evidencia
RN41	Media	El usuario autenticado consulta su propio perfil.	**Implementada para consulta.** `GET /api/auth/perfil` usa la identidad autenticada; edición de perfil es otra función.
RN42	Media	Cualquier usuario autenticado consulta especialistas.	**Implementada.** Ruta está bajo el middleware global; no exige rol más específico.
RN43	Media	Cualquier usuario autenticado busca usuario por ID.	**Implementada literalmente, requiere revisión de privacidad.** La ruta no limita campos/propiedad y puede devolver datos personales. Definir qué atributos pueden exponerse.
RN44	Media	Filtrar citas por estado, fecha, mascota y especialista.	**Implementada.** Esquemas de query y consultas de listado.
RN45	Media	Filtrar disponibilidad por usuario, fecha y estado.	**Implementada.** Esquemas de query y consultas de disponibilidad.

6.8 Reglas complementarias identificadas en el proyecto

Estas reglas no estaban enumeradas en el archivo RN adjunto. Se identificaron en `fechaHora.js`, los servicios/modelos de disponibilidad y citas, el proceso `citaCancellationScheduler.js` y la bitácora de modernización. Conviene incorporarlas al catálogo oficial del equipo.

ID	Prioridad sugerida	Regla complementaria	Estado observado / evidencia
RN46	Alta	No crear ni mover una franja de disponibilidad a una fecha/hora pasada, usando la zona `America/Bogota`.	**Implementada.** Se valida en backend y se refleja en la interfaz.
RN47	Alta	Reservar o reprogramar citas solo en una franja futura; el backend vuelve a validar la franja dentro de la transacción para evitar condiciones de carrera.	**Implementada.** Filtro `futuras=true`, comprobación de servicio y bloqueo en la operación transaccional.
RN48	Alta	Al iniciar el backend y cada 60 segundos, cancelar citas `pendiente` o `confirmado` cuyo horario ya pasó en `America/Bogota`.	**Implementada.** Worker programado; procesa solo los estados definidos y vuelve a intentar vencimientos al reiniciar.
RN49	Alta	Registrar una sola auditoría por cancelación, con actor/rol, origen, motivo y fecha; actualizar estado y auditoría de forma atómica.	**Implementada.** Tabla `cita_cancelacion`, transacción y restricción única por cita; las cancelaciones automáticas usan actor `sistema`.
RN50	Media	Conservar las citas vencidas y sus franjas como historial; no purgar automáticamente disponibilidad antigua. Una franja vencida cancelada automáticamente no se vuelve a ofrecer.	**Implementada.** El proceso cancela, no elimina; la disponibilidad antigua permanece y las referencias históricas se conservan.

7. Seguimiento del plan de 18 tareas

El PDF conserva el estado inicial “Por hacer” para sus tareas. La columna de abajo es una evaluación estática separada basada en el código presente en esta copia, a fecha de revisión; no cambia retrospectivamente el contenido del plan ni sustituye pruebas dinámicas.

N.º	Prioridad	Tarea del plan	Estado observado en el proyecto
1	P0	Backend: usar ESM	**Implementada.** `type: module` y módulos ESM.
2	P0	Backend: migrar a Vitest	**Implementada.** Vitest/Supertest configurados; hay pruebas unitarias e integración.
3	P0	Manejo de errores sin duplicidad, logs seguros, tipado y normalización de espacios	**Implementada en lo esencial.** Error handler único, Pino con redacción, Joi, normalización y reglas de espacios; JavaScript mantiene tipado dinámico (TypeScript se usa en frontend).
4	P0	Backend: migrar a Prisma	**Implementada.** Prisma 7 ESM, cliente, esquema y migraciones.
5	P0	Encriptar contraseñas con bcrypt	**Implementada.** bcrypt en altas/nuevos hashes; compatibilidad de lectura/migración para hashes scrypt heredados.
6	P0	Evitar SQL Injection	**Implementada en el acceso revisado.** Consultas parametrizadas y pruebas específicas; no usar APIs `Unsafe` con entrada externa.
7	P0	Limitar intentos de login	**Pendiente.** No se encontró rate limiter configurado para autenticación.
8	P0	Cierre de sesión que invalida tokens	**Parcial.** El frontend limpia sesión local; no se encontró endpoint/lista de revocación en backend.
9	P0	Refactorizar Dockerfiles	**Por confirmar.** Dockerfiles y Compose funcionales existen; no se encontró evidencia de cierre que demuestre que la tarea específica de refactor se completó.

N.º	Prioridad	Tarea del plan	Estado observado en el proyecto
10	P0	Quitar valores por defecto de JWT_SECRET y credenciales de BD	**Parcial.** El módulo JWT falla si no recibe secreto, pero Compose inyecta placeholder; `DB_PASSWORD` también tiene fallback.
11	P0	Configurar CORS correctamente	**Parcial.** Express lee `CORS_ORIGIN`, pero Compose usa `*` como default y el código espera una lista literal de orígenes. Configurar dominios exactos.
12	P1	Maquetado completo: header, contenido y footer	**Parcial.** Hay Navbar y layout principal con `<main>`; no se encontró footer general.
13	P1	Recuperar contraseña por correo	**Pendiente.** No se encontraron endpoints/UI de reset ni envío de correo.
14	P1	Rediseñar navegación por secciones con íconos	**Parcial.** Hay navegación por rol e iconos en partes de la interfaz; la Navbar no implementa todavía la reorganización completa descrita.
15	P1	Pantallas/estados de carga	**Parcial.** Algunas vistas comunican “Cargando…” y deshabilitan acciones; no hay un patrón global y uniforme.
16	P1	Historias clínicas desde la vista especialista	**Parcial.** API y servicios clínicos existen; `CitasEspecialista` muestra citas, pero la revisión no encontró el flujo completo de historia clínica integrado en esa vista.
17	P1	Personalizar perfil de usuario y mascota	**Parcial.** Existe consulta de perfil y alta de mascota; no se encontró el flujo completo de edición de perfil de usuario/mascota.
18	P1	Cuidador con seguimiento de mascota	**Pendiente.** No se encontró modelo, endpoint ni pantalla de seguimiento/cuidado.

8. Contradicciones y riesgos que deben mantenerse visibles

1. **Headers frente a JWT:** RN05/RN06 y parte de Swagger aún hablan de `x-user-id` / `x-user-role`; el código protege con JWT Bearer y obtiene rol de la BD. Actualizar el documento RN y la especificación Swagger. No habilitar identidad confiando en headers libres.
2. **Modelos:** documentos heredados dicen siete modelos; el schema actual contiene ocho por `CitaCancelacion`.
3. **Hash de contraseñas:** algunas secciones antiguas del README dicen scrypt; el código actual genera bcrypt y solo mantiene compatibilidad/migración de hashes scrypt heredados.
4. **Auditoría:** hay auditoría persistente para cancelaciones, no un ledger general de CRUD. El middleware general no persiste en producción.
5. **Credenciales/CORS:** defaults inseguros en Compose deben retirarse o exigir configuración no vacía. Configurar `CORS_ORIGIN` con orígenes explícitos y comprobar preflight.
6. **Límite de login y revocación:** no se encontraron. Son pendientes P0 del plan.
7. **Datos personales:** usuarios autenticados pueden consultar rutas por ID que pueden exponer correo, teléfono o dirección. Revisar minimización de datos y permisos conforme a la política de privacidad del producto.
8. **Swagger y README:** algunos contenidos están desactualizados respecto de JWT, cantidad de modelos, hashing y configuración CORS. Las pruebas de frontend/E2E y un pipeline CI siguen siendo brechas de calidad identificadas en los documentos del proyecto.
9. **Producción:** el README caracteriza el proyecto como desarrollo/demo y no listo para producción. Antes de desplegar públicamente se necesitan secretos gestionados, CORS restringido, rate limiting, política de sesión, privacidad, backups/restauración y pruebas de seguridad operativa.

9. Próximos pasos documentales y técnicos sugeridos

1. Actualizar RN05/RN06 para describir JWT Bearer y rol consultado desde PostgreSQL.
2. Alinear Swagger con JWT Bearer y respuestas reales; retirar headers de identidad y `mock-token`.
3. Completar los pendientes P0: límite de login, estrategia de revocación y eliminación de defaults de secretos/credenciales; validar CORS con orígenes reales.
4. Decidir si la auditoría de acciones administrativas debe persistirse en una tabla general y qué retención tendrá.

5. Definir las reglas de exposición de datos personales en consultas por ID.
6. Antes de añadir tablas futuras, acordar historias de usuario, permisos, campos, expiración/retención y criterios de borrado; crear migraciones aditivas y pruebas.
7. Añadir pruebas de frontend y E2E a medida que se completen las tareas P1; documentar build y ejecución en CI.

10. Referencias dentro del repositorio

- Esquema de datos: `back/prisma/schema.prisma`.
- Migraciones: `back/prisma/migrations/`.
- Configuración de Prisma: `back/app/config/prisma.js`, `back/prisma.config.ts`.
- Rutas y servicios: `back/app/routes/`, `back/app/services/`.
- Identidad y permisos: `back/app/middlewares/identifyUser.js`.
- Errores, logs y respuesta: `back/app/middlewares/errorHandler.js`, `back/app/utils/logger.js`, `back/app/utils/response.js`.
- JWT y contraseñas: `back/app/utils/jwt.js`, `back/app/utils/password.js`.
- Citas y cancelación automática: `back/app/models/cita.model.js`, `back/app/services/citaCancellationScheduler.js`.
- Frontend: `front/src/routes/AppRoutes.tsx`, `front/src/layouts/MainLayout.tsx`, `front/src/context/AuthContext.tsx`, `front/src/services/api.ts`.
- Arquitectura complementaria: `docs/back/ARQUITECTURA.md`.
- Bitácora Prisma y citas: `docs/back/BITACORA_MODERNIZACION_PRISMA_Y_CITAS.md`.
- Bitácora de validación de formularios: `docs/back/BITACORA_VALIDACION_FORMULARIOS.md`.
- Fuentes originales adjuntas a la conversación (no copiadas al repositorio): `Reglas_de_Negocio_RN.docx` y `Plan_18_Tareas_Huellitas_Saludables_Detallado.pdf`.

Mantenimiento: cada cambio de una regla de negocio debe actualizar su ID/estado, el servicio y las restricciones que correspondan, las pruebas y esta tabla. Las reglas de autorización deben tener verificación en backend aunque también se oculten opciones en el frontend. Las decisiones nuevas deben añadir evidencia, consecuencias, fecha y estado sin presentar como histórica una justificación que no esté documentada.